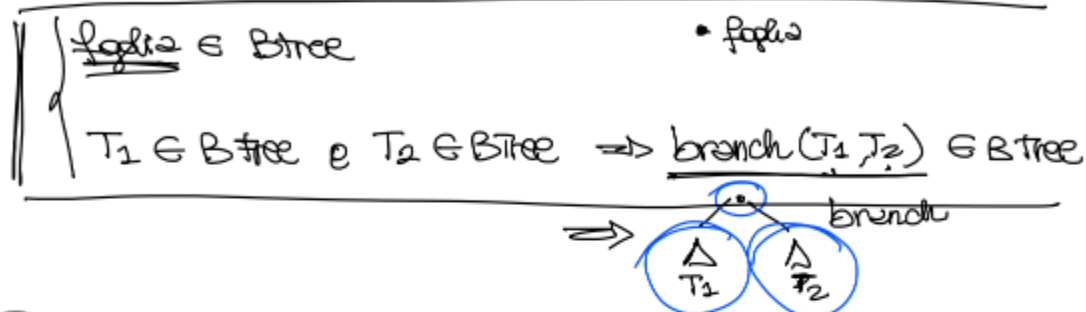


INDUZIONE STRUTTURALE

Definiamo BTree come



Definiamo $m : \text{BTree} \rightarrow \mathbb{N}$ conta i nodi di un albero binario

$$\begin{cases} m(\text{folia}) = 1 \\ m(\text{branch}(T_1, T_2)) = 1 + m(T_1) + m(T_2) \end{cases}$$

Definiamo $h : \text{BTree} \rightarrow \mathbb{N}$ calcola l'altezza dell'albero

$$\begin{cases} h(\text{folia}) = 0 \\ h(\text{branch}(T_1, T_2)) = 1 + \max(h(T_1), h(T_2)) \end{cases}$$

calcola il valore massimo tra due valori

Dimostrare per induzione ~~strutturale~~ strutturale che

$$(*) \quad \forall T \in \text{BTree}. m(T) \leq 2^{h(T)+1} - 1$$



Base: Dobbiamo dimostrare (*) per gli elementi folia

$$T = \text{folia} \quad m(T) = m(\text{folia}) = 1$$

$$h(T) = h(\text{folia}) = 0$$

$$\text{Calcoliamo } 2^{h(T)+1} - 1 = 2^{0+1} - 1 = 1$$

$$M(T) = 2^{h(T)+1} - 1 \quad \checkmark$$

Poss. induttivo : $T = \text{branch}(T_1, T_2)$

Supponiamo per Hp. ind. che

$$\text{Hp. ind.} \rightarrow \begin{cases} \textcircled{*} M(T_1) \leq 2^{h(T_1)+1} - 1 \\ \textcircled{*} M(T_2) \leq 2^{h(T_2)+1} - 1 \end{cases}$$

dimostriamo che $M(T) \leq 2^{h(T)+1} - 1$

$$M(T) = 1 + M(T_1) + M(T_2) \quad \boxed{h(T)} = \underbrace{(1 + \max(h(T_1), h(T_2)))}_{= h}$$

$$M(T) = 1 + M(T_1) + M(T_2) \leq 1 + 2^{h(T_1)+1} - 1 + 2^{h(T_2)+1} - 1$$

per def di M

per $\textcircled{*}$

$$\text{Hp. ind.} \rightarrow 2(2^h) = 2^{h+1}$$

$$\leq 2^h + 2^h - 1$$

$$= 2^{h+1} - 1$$

$$= 2^{h(T)+1} - 1$$

$$h \geq h(T_1) + 1$$

$$h \geq h(T_2) + 1$$

BTree, definiamo

$\ell : \text{BTree} \rightarrow \mathbb{N}$ conta le foglie

$$\ell(\text{foglia}) = 1$$

$$\ell(\text{branch}(T_1, T_2)) = \ell(T_1) + \ell(T_2)$$

Dimostrare che $\ell(T) \leq 2^{h(T)} \quad \forall T \in \text{BTree}$

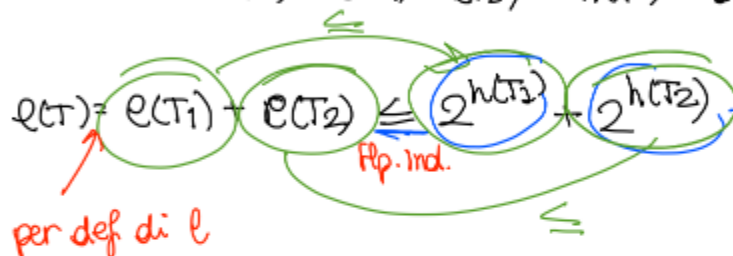
Base: $T = \text{foglia}$: $e(T) = 1$ $1 \leq 2^0 = 1$ ✓
 $h(T) = 0$

Passo induttivo $T = \text{branch}(T_1, T_2)$

Hp. ind.: $e(T_1) \leq 2^{h(T_1)}$ $e(T_2) \leq 2^{h(T_2)}$

Dobbiamo dimostrare che $e(T) \leq 2^{h(T)}$

$$e(T) = e(T_1) + e(T_2) \quad h(T) = 1 + \max(h(T_1), h(T_2))$$



$$h = \max(h(T_1), h(T_2))$$

$\Rightarrow$

$$h(T_1) \leq h$$

$$h(T_2) \leq h$$

Exp (espressioni) definite induttivamente
 aritmetiche

$\begin{cases} m \in \text{Exp} \\ e_1, e_2 \in \text{Exp} \end{cases}$ def. di exp
 $e_1 + e_2 \in \text{Exp} \quad e_1 - e_2 \in \text{Exp}$

Def. val: $\text{Exp} \rightarrow \mathbb{N}$ numero di numeri m presenti in una exp

$$\begin{cases} \text{val}(m) = 1 \\ \text{val}(e_1 + e_2) = \text{val}(e_1 - e_2) = \text{val}(e_1) + \text{val}(e_2) \end{cases}$$

Def op: $\text{Exp} \rightarrow \mathbb{N}$ numero di operatori in exp $(+, -)$

$$\begin{cases} \text{op}(m) = 0 \\ \text{op}(e_1 + e_2) = \text{op}(e_1 - e_2) = \text{op}(e_1) + \text{op}(e_2) + 1 \end{cases}$$

Dimostrare che $val(e) = op(e) + 1 \quad \forall e \in Exp$

Base: $e = m \quad val(e) = val(m) = 1$
 $op(e) = op(m) = 0 \quad 1 = 0 + 1 \quad \checkmark$

Passo ind: $e_1 + e_2$ ($e_1 - e_2$ analogo)
 perché la definizione di
 op e val non dipende
 dal tipo di operatore

Hip. ind $val(e_1) = op(e_1) + 1$
 $val(e_2) = op(e_2) + 1$

Dobbiamo dim. che $val(e) = op(e) + 1$

$$val(e) = \underset{\substack{\uparrow \\ \text{per def di val}}}{val(e_1)} + \underset{\substack{\uparrow \\ \text{H.p. ind.}}}{val(e_2)} = \underset{\substack{\uparrow \\ \text{H.p. ind.}}}{op(e_1) + 1} + \underset{\substack{\uparrow \\ \text{H.p. ind.}}}{op(e_2) + 1}$$

$$= \boxed{op(e_1) + op(e_2) + 1} + 1 = op(e) + 1$$

$op(e)$
per def di op

```

int x=1;
int y=2;
void A() { int z = x+y; }
void B() { int x=2;
           int w=4;
           void C() { int z = y+w;
                     A();
                     }
           y = x+y;
    }
    
```

main

$\begin{bmatrix} x \\ y \end{bmatrix}$	A	$\begin{bmatrix} z \end{bmatrix}$	$Sol(main) = 0$
$\begin{bmatrix} x \\ y \end{bmatrix}$	B	$\begin{bmatrix} z \\ w \end{bmatrix}$	$Sol(A) = Sol(B) = 1$
$\begin{bmatrix} x \\ y \end{bmatrix}$	C	$\begin{bmatrix} z \end{bmatrix}$	$Sol(C) = 2$

B();
{

main $\rightarrow B \rightarrow C \rightarrow A$

Scoping statico

main

x	1
y	2

Chiamata main $\rightarrow B$

main

x	1
y	2

link dinam.

x	2
w	4

chiamante chiamato

$$K_B = \text{Sd}(\text{main}) - \text{Sd}(B) + 1$$

$$= 0 - 1 + 1 = 0$$

$$CS(B) = \text{main}$$

Esecuzione di B

base = 2

$$y = x + y$$

$$N_y = -\text{Sd}(\text{main}) + \text{Sd}(B)$$

$$\Rightarrow y_{\text{main}} = 2$$

$$= 0 + 1 = 1$$

$$y = 2 + 2 = 4$$

new main

la più vicina nella catena di domini che def. y
uso y
Dobbiamo risolvere di 1 la catena statica per trovare il corretto riferimento a y

Chiamata B $\rightarrow C$

main

x	2
y	4

B

x	2
w	4

C

x	8
---	---

Link statico:

$$K_C = \text{Sd}(B) - \text{Sd}(C) + 1$$

$$= 1 - 2 + 1 = 0$$

$$CS(C) = B$$

base

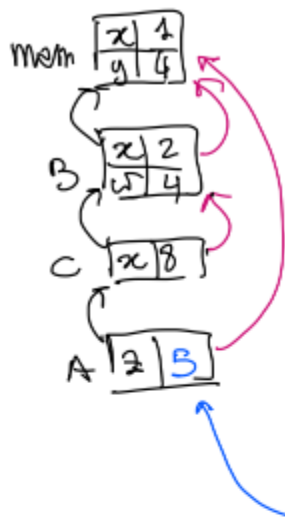
$$x = y + w$$

$$N_w = \text{Sd}(C) - \text{Sd}(B)$$

$$= 1 \text{ in } B \rightarrow 4$$

$$N_y = \text{Sd}(\text{main}) - \text{Sd}(C) = 2 \text{ in main}$$

Chiamata $C \rightarrow A$



Calcolo storico

$$K_A = \text{Sol}(C) - \text{Sol}(A) + 1$$

$$= 2$$

$$CS(A) = CS(CS(C)) =$$

$$= CS(B) = \text{main}$$

base

$$z = x + y$$

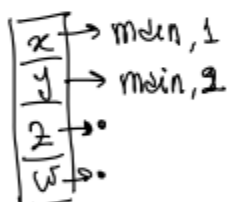
$\downarrow$ $\downarrow$
 1 4

$$N_y = N_z = \text{Sol}(A) - \text{Sol}(\text{main})$$

$$= 1 \rightarrow \text{main}$$

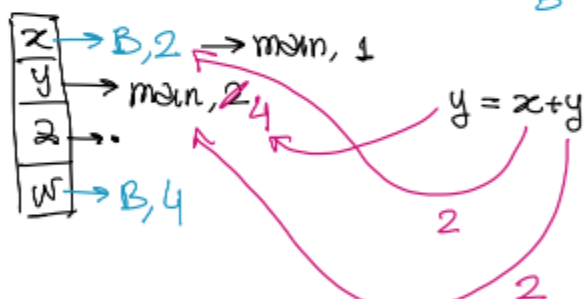
Scoping dynamics

CRT



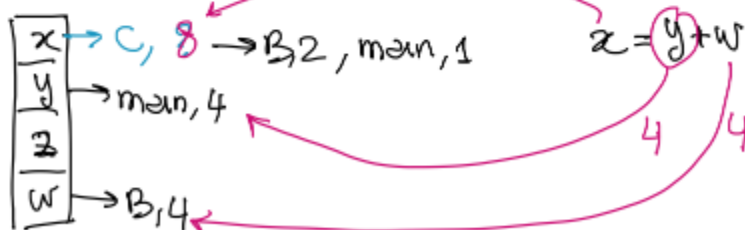
Chiamata $\text{main} \rightarrow B$

Esecuzione B

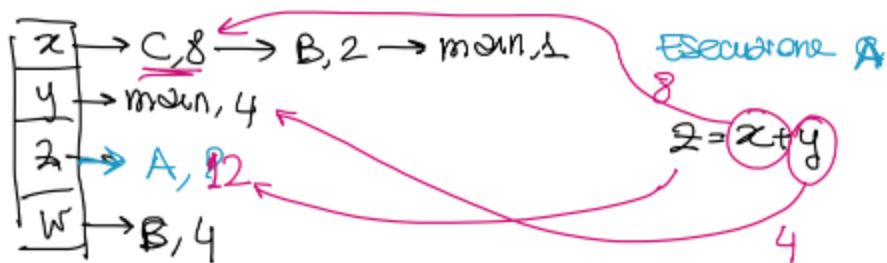


Chiamata $B \rightarrow C$

Esecuzione C



Chiamata $C \rightarrow A$



Ricorsione

lista function (lista list) {

if (length(list) == 0) then return list == ()

else return concat (function (cdr (list)), car (list))

}

function(3, 6, 9) → (9, 6, 3)

↳ concat (function(6, 9), (3))

↳ concat (function(9), 6)

↳ concat (function(()), 9)

La funzione inverte gli elementi della lista

lista functionrc (lista list, lista res) {

if length(list) == 0 return res;

else return functionrc (cdr (list), concat (car (list), res));

}

lista function (lista list) { return functionrc (list, ()); }

function (3, 6, 9)

↳ functionrc (369, ())

↳ functionrc (69, concat(3, ()))

↳ functionrc (9, concat(6, (3)))

↳ functionrc (, concat(9, (63)))

(9, 6, 3)